

Project 1: This Ship is on Fiiiiire!

Rutgers University | CS 440 (01:198:440) | Fall 2026
Yaksh Bhatia (ysb19) and Vraj Raj (var88)

Overview

We tested four bots on randomly generated ships to see how well they could reach the button while avoiding a spreading fire. Bots 1-3 use the strategies from the assignment. For Bot 4, we estimate how dangerous each cell will be when the bot reaches it. The bot uses these estimates to choose a route, but can switch permanently to the shortest path when taking a detour does not seem worth it.

In the dense sweep, Bot 4 succeeded in 83.64% of runs, compared with 83.04% for Bot 3, 82.74% for Bot 2, and 80.87% for Bot 1. On the same setups, Bot 4 won 357 cases that Bot 3 lost, but lost 166 that Bot 3 won. The improvement was small and mostly appeared in the middle range of flammability. At high flammability, the bots performed almost the same. Overall, looking ahead helped, but being more careful did not always lead to a better result.

1. Environment and simulation

Ship generation - Yaksh Bhatia. We start with a blocked D by D grid and open a random interior cell. We then repeatedly pick a random blocked cell with exactly one open neighbor and open it, stopping when no such cells remain. Neighbors are only up, down, left, and right. This gives us a connected tree of open cells. Next, we count the dead ends, N_0 . We repeatedly pick a random dead end and open one of its random blocked neighbors until 2 times the remaining dead-end count is at most N_0 . This second part adds loops to the ship.

Fire and simulation - Vraj Raj. We place the bot, button, and initial fire on three different random open cells. A non-burning open cell with K burning neighbors catches fire with probability $1 - (1 - q)^K$. We update the fire synchronously: first we decide which cells ignite using the old fire state, then we apply all those changes together. Fire cannot enter walls, and burning cells stay on fire.

Each timestep starts with the bot choosing and making a legal move. If it steps into a burning cell, it fails immediately. If it reaches the button safely, it succeeds before the fire spreads again. Otherwise, the fire spreads and we check whether the bot or button burned. This order is important because the bot does not need to survive another fire update after reaching the button.

1. Setup, assumptions, and baseline bots

Writing responsibility: Yaksh Bhatia: ship and Bots 1-2; Vraj Raj: fire, simulation, and Bot 3

Implementation choices

We only require the first opened cell to be in the interior, so border cells can open later. We allow the button to burn because it is on an ordinary open cell. If that happens, we record `BUTTON_BURNED`. If the bot and button burn during the same update, we record `BURNED`. The bot knows q and is allowed to stay in place. The simulator raises an error for an illegal move.

A bot returns `GAVE_UP` if it cannot reach the button through non-burning cells. This does not give up a possible future success, since the fire never goes out and a blocked route cannot reopen. Bot 1 can give up before its first move if it cannot find an initial path. We set a step limit of $4D^2$, but neither sweep had a `TIMEOUT`.

Bot 3 also applies its buffer to the button. If that prevents it from finding a route, it tries again while avoiding only the burning cells. Our BFS helper lets the bot start inside the buffer, since it is already there and needs a chance to move away.

The required strategies

Bot	Decision rule
Bot 1	Compute a shortest path once, avoiding the initial fire cell. Follow it without reacting to later fire.
Bot 2	At every move, use BFS to find a shortest path avoiding all cells currently on fire.
Bot 3	At every move, use BFS to avoid fire and its adjacent cells. If no such path exists, use Bot 2's rule.

All three bots use the same BFS helper. BFS finds a shortest path because every move has the same cost. We check neighbors in the same up/down/left/right order for all bots, so ties are handled consistently. Bot 4 also uses this helper to find its shortest-path option.

Keeping the simulation efficient and repeatable

To speed up ship generation, we keep track of each cell's open-neighbor count and update only the affected cells after an opening. We do not rescan the whole grid each time. The candidate and dead-end sets use a list and an index dictionary, so random selection and swap-removal take constant time. We update every affected dead end, since one opening can change several of them. We also compute the open-cell neighbor lists once and reuse them.

For fire updates, we track the non-burning cells next to fire and how many burning neighbors they have. We call this the frontier. Sorting the frontier and taking one random draw per cell makes the fire reproducible. We only test frontier cells for ignition, although sorting them takes $O(F \log F)$ time for frontier size F . Each bot sees the same fire in a matched (q , trial) case for as long as its run continues. The bots do not use the real fire's random generator to decide their moves.

Our project notes report about 20 ms to generate a $D = 100$ ship. For 50 ships at $D = 40$, they report about 65.5% open cells, with dead ends dropping from about 290 to 144. These measurements come from the handoff; the sweep CSVs do not store generation benchmarks.

2. Q1: Predicting danger before moving

Writing responsibility: Yaksh Bhatia: risk model and arrival-time calculation

We named Bot 4 the Travis Scott Bot in the code. Instead of just avoiding cells next to fire like Bot 3, it estimates future burn probabilities. It uses the ship layout, current fire, bot and button positions, and q . We write $p_s(v)$ for the estimated chance that cell v is burning after s more fire steps. At the start, $p_0(v)$ is 1 for burning cells and 0 for the others.

For each open neighbor u , we use $q p_s(u)$ as its estimated chance of spreading fire to the cell. Multiplying $(1 - q p_s(u))$ over the neighbors gives the estimated chance that none spread fire. We also account for the cell already being on fire:

$$p_{s+1}(v) = p_s(v) + [1 - p_s(v)] [1 - \text{product}_{u \in N(v)} (1 - q p_s(u))].$$

If every neighbor is either definitely burning or definitely safe, the product becomes $(1 - q)^K$. This means our first prediction matches the assignment's fire rule. After that, the model treats related fire events as if they were independent. This makes the calculation manageable, but the later predictions are only approximations.



Figure 1. Risk predictions at $q = 0.3$ after 6 fire steps. The predicted fire reaches farther through the corridors at 10 steps than at 3. Darker orange means a higher predicted burn chance, and dark gray shows walls. This shows how the model works, but does not test how accurate its predictions are.

Arrival time and the button deadline

If the bot reaches an ordinary cell v on its m -th future move, we use $d_m = p_m(v)$ as the danger. This covers both stepping into existing fire and getting caught during that move's fire update. For the button, we use $p_{m-1}(v)$, since reaching it ends the trial before fire spreads. We estimate a path's success chance by multiplying $(1 - d_m)$ over the cells the bot will visit.

Including the button's risk penalizes detours that keep the bot safe but take too long. Beyond the prediction horizon, we reuse the last forecast, which can underestimate later danger. The path calculation also assumes the survival events are independent, so it is only an estimate.

2. Q1: Routing and the FE!N decision

Writing responsibility: Yaksh Bhatia: path search; Vraj Raj: switching rule and configuration

Length plus estimated risk

We use a heap-based search similar to Dijkstra's algorithm to find a route that balances length and risk. Currently burning cells are never allowed. For a candidate cell reached on move m , we add the cost below. Here, w is `risk_weight` and d is the predicted danger when the bot arrives:

$$\text{edge cost} = 1 + w [-\log(1 - \min(d, 0.999))].$$

Without the cap, adding the negative logs gives the negative log of the estimated chance of surviving the whole path. This lets us combine path length and risk in one cost. We cap danger so that the log stays finite, even when the model predicts certain fire. When we later compare the paths' success chances, we use the original, uncapped probabilities.

The heap also stores how many moves it takes to reach a cell, because arriving later can be more dangerous. However, we keep only the cheapest entry for each cell. This can throw away a more expensive route that arrives earlier and would be better afterward. Because of this shortcut, our search is a heuristic and is not guaranteed to find the best route for these time-dependent costs.

When the bot switches to FE!N Mode

At each step, Bot 4 first finds the shortest path that avoids current fire. It gives up if there is no such path. Otherwise, it updates the risk predictions. In Risk-Aware Mode, it also finds a careful route and compares the estimated success chances of the two options.

Bot 4 switches to FE!N Mode if it cannot find a careful route. It also switches if the careful route is longer and either its estimated improvement is below `fein_margin` or it needs more than `max_detour_ratio` times as many moves as the shortest path. If the careful path is no longer, the bot stays in Risk-Aware Mode. Once FE!N starts, it stays on. The bot then recomputes and follows a Bot 2 shortest path at every step.

Setting	Value used	Purpose
<code>risk_weight</code>	30	Weight on the negative-log survival cost
<code>horizon</code>	20	Number of future fire steps predicted
<code>fein_margin</code>	0.02	Required survival-probability gain for a detour
<code>max_detour_ratio</code>	2.0	Maximum accepted careful/shortest move ratio

We initially used a risk weight of 3. As the handoff explains, this made a substantial chance of dying cost only about one extra move, so the bot rarely took detours. We raised it to 30. The main success curves use the defaults above. The tuning results on page 6 show small differences between settings, but do not establish one best configuration.

Keeping Bot 4 efficient

We build the predictions outward from the current fire and save the result for each future step. This avoids looping over the whole grid each time, although the area we need to check can still get large. The path search uses a heap, and BFS reuses the saved neighbor lists. FE!N Mode skips the risk-weighted search, but it still calculates the risk map for the logs, so it is not as cheap as Bot 2.

2. Q1: Debugging and runtime

Writing responsibility: Yaksh Bhatia: risk-model efficiency; Vraj Raj: trigger diagnosis

A switching bug changed the bot's behavior

Our first FE!N rule switched modes whenever the careful path did not improve estimated survival by at least 0.02. This caused a problem when the fire started far away: the careful and shortest paths were often the same, so the estimated improvement was zero. The bot would switch on its first move and never return to risk-aware planning.

The handoff records 28,041 of 32,000 Bot 4 runs entering FE!N on the first move in the original dense experiment, about 88%. We fixed this by requiring a real detour before checking the improvement and length limits. In the fixed dense results, 4,242 of 32,000 runs enter FE!N at some point, about 13%. The first count is for first-move switches, while the second includes switches at any time. The old CSV is not included. All results here use the fixed bot.

Measured computation cost

Bot	Time per trial	Time per step
Bot 1	3.6 ms	0.12 ms
Bot 2	5.8 ms	0.19 ms
Bot 3	6.2 ms	0.20 ms
Bot 4	53.7 ms	1.73 ms

Table 1. Timings from the handoff for $D = 40$, $q = 0.3$, 200 trials, and one core. These were not remeasured and are not stored in the sweep CSVs. Bot 4 takes about 9 times as long per step as Bot 3.

Bot 4 does more work for a small improvement over Bot 3. This trade-off would be more important if the fire kept spreading while the bot was thinking. Our simulator waits for the bot to choose a move, so the success rates do not include any cost for taking longer to decide.

Limitations of the current design

There are several ways the design can make a poor choice. The fire predictions and path-success calculation ignore correlations. Risk stops increasing after the prediction horizon. Keeping one search entry per cell can discard useful arrival times. Also, once FE!N starts, the bot cannot go back to a careful route even if that would help later.

What we learned from tuning

Figure 2 shows that changing the settings made only small differences in our tests. A higher risk penalty or a longer forecast did not always help. For example, horizon 40 gives 2,515 successes, compared with 2,520 for horizon 20 and 2,525 for horizon 10. One possible reason is that prediction errors build up farther into the future, but these results alone do not prove that explanation.

A smaller FE!N margin keeps the bot in Risk-Aware Mode more often. With margin 0, FE!N activates in 153 of 3,000 cases, compared with 396 at the default 0.02. It also gives 6 more successes. This suggests that the default threshold may switch to rushing too early in some cases, but we would need fresh test setups to check that. We did not record tuning runtimes, so we cannot measure how much time a shorter horizon would save.

2. Q1: Bot 4 parameter tuning

Writing responsibility: Yaksh Bhatia: risk_weight and horizon; Vraj Raj: FEIN margin and detour ratio

We varied one parameter at a time, keeping the others at their defaults. Each value uses 500 matched setup IDs at each $q = 0.1, 0.2, 0.3, 0.4, 0.5$, and 0.6 , with $D = 40$ and seed 440. Four settings per parameter produce 12,000 runs per sweep and 48,000 runs across all four sweeps.

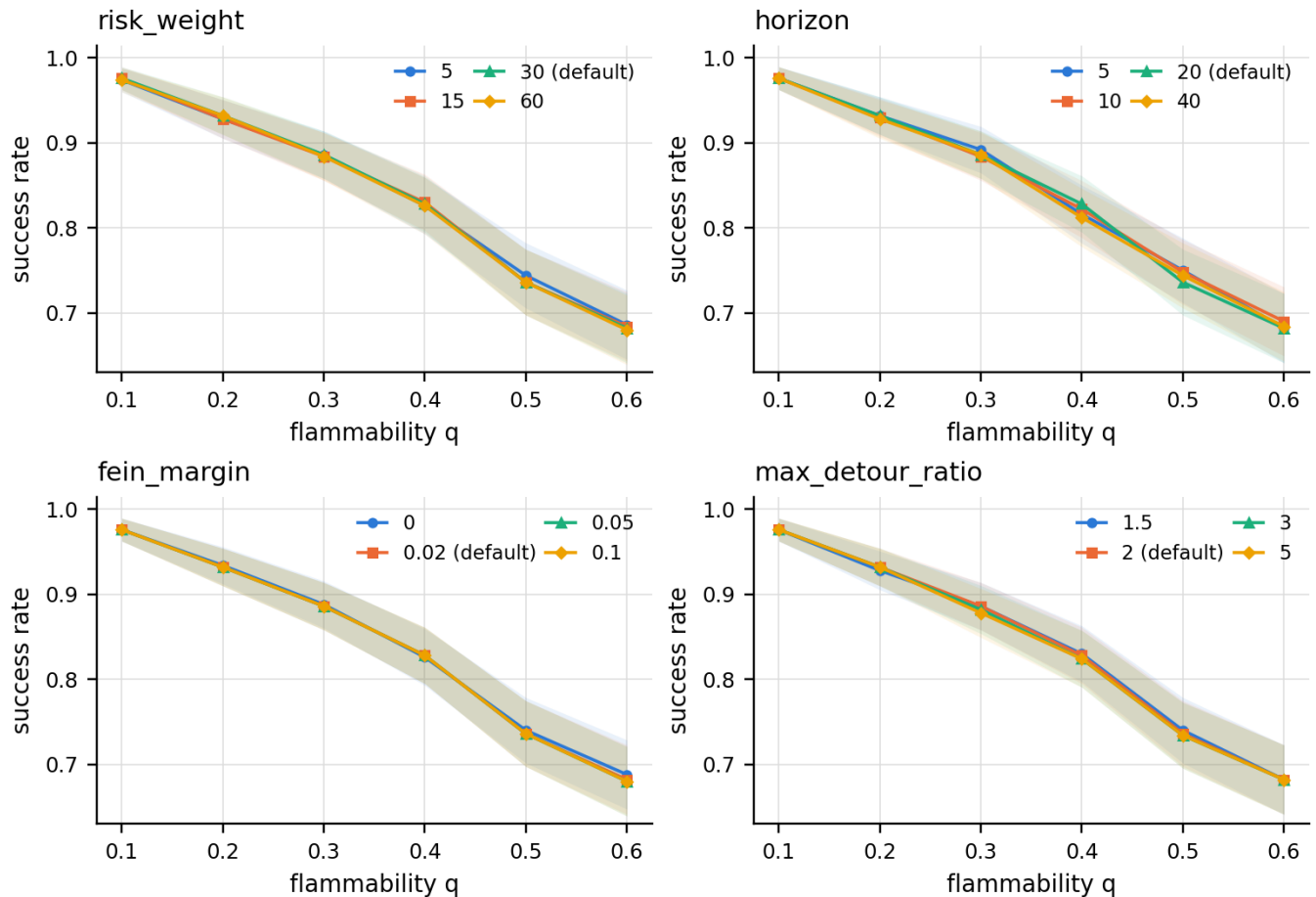


Figure 2. All four tuning sweeps, redrawn from tuning/results/*.csv. Each point has 500 runs; shading is the same pointwise normal-approximation 95% interval used in the main curves. Legends give the tested values and identify the default. The curves overlap closely.

Parameter	Best sampled value	Successes / 3,000	Wins / losses vs. default
risk_weight	5	2,522 (84.07%)	10 / 8
horizon	10	2,525 (84.17%)	18 / 13
fein_margin	0	2,526 (84.20%)	7 / 1
max_detour_ratio	1.5	2,521 (84.03%)	5 / 4

The default has 2,520 successes (84.00%) in every sweep. Wins and losses compare different outcomes on the identical (q , trial) case. Across tested values, totals range from 2,516-2,522 for risk weight, 2,515-2,525 for horizon, 2,519-2,526 for margin, and 2,513-2,521 for detour ratio.

These settings need more testing. Setup IDs repeat across q and overlap the main sweeps; default results match the dense-sweep rows exactly. We chose the best values on these same cases and did not test them together or on new setups. The four-bot comparison keeps the original defaults.

3. Q2: Experiment design and the full range

Writing responsibility: Yaksh Bhatia

We used $D = 40$ and base seed 440 for the experiments. A trial ID determines the ship, starting positions, and fire seed. For a given (q, trial) , all four bots face the same fire. We also reuse trial IDs across q , so the layout and starting positions stay the same while flammability changes.

Sweep	Flammability values	Trials per q	Bot runs
Coarse	0.00 to 1.00, step 0.05	200	16,800
Dense	0.04 to 0.66, step 0.02	1,000	128,000

The coarse sweep showed that the bots differed most around $q = 0.05$ to 0.65 , so we ran more trials in that range. We chose $D = 40$ to keep Bot 4's runtime manageable while still collecting many trials. The handoff reports using eight worker cores, with about 40 seconds for the coarse sweep and about 5-7 minutes for the dense sweep. We have not tested whether the same results hold on larger ships.

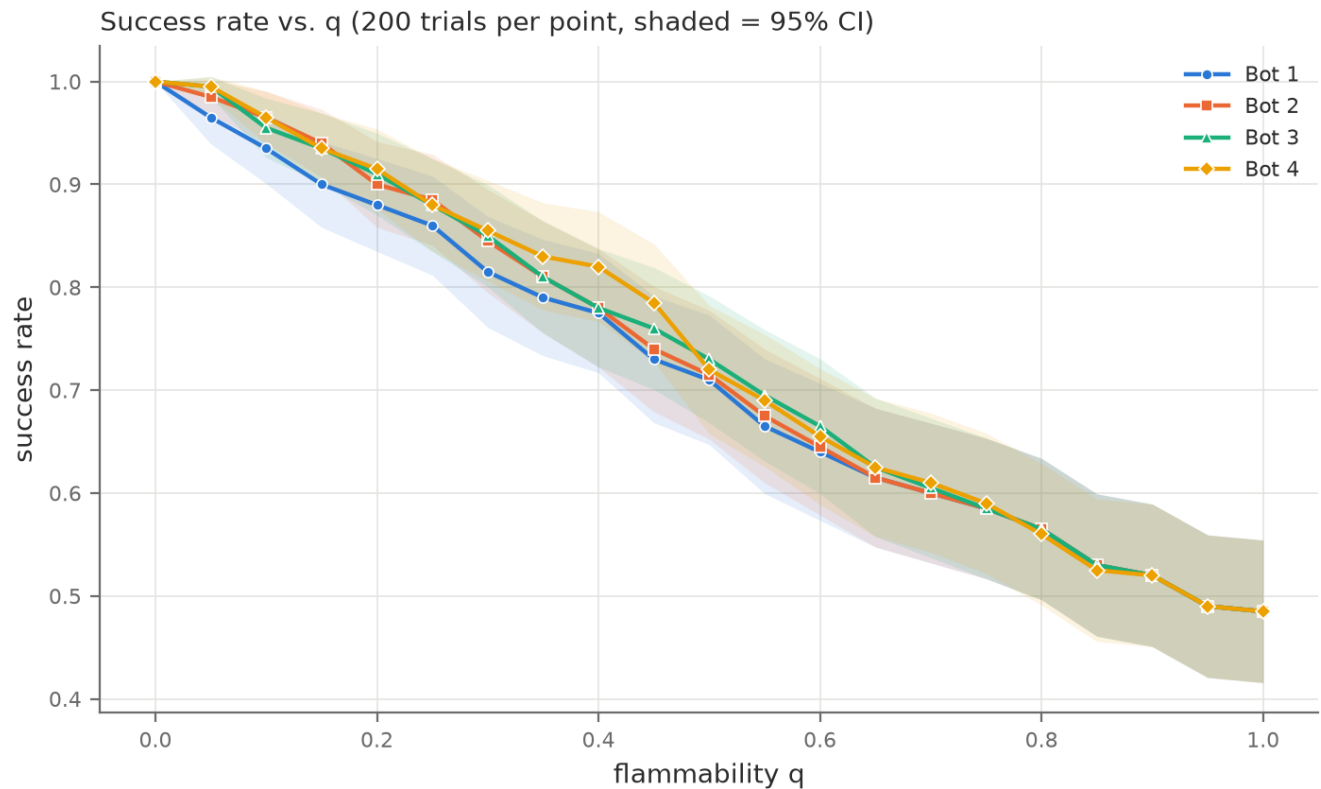


Figure 3. Full-range success rates from results/coarse.csv, with 200 matched setups per q and bot. All four bots succeed on the sampled $q = 0$ cases. Differences shrink at high q , and all reach 48.5% success at $q = 1$.

The shaded bands show a normal-approximation 95% confidence interval at each point: $p \pm 1.96 \sqrt{p(1-p)/n}$. For p near 0.8 and $n = 1,000$, this is about 2.5 percentage points on either side. This approximation works poorly near 0 or 1. In particular, getting success in every trial gives a zero-width band, but does not prove the true success probability is exactly 1. These intervals apply to individual points, not the whole curve at once.

3. Q2: Results in the interesting range

Writing responsibility: Yaksh Bhatia

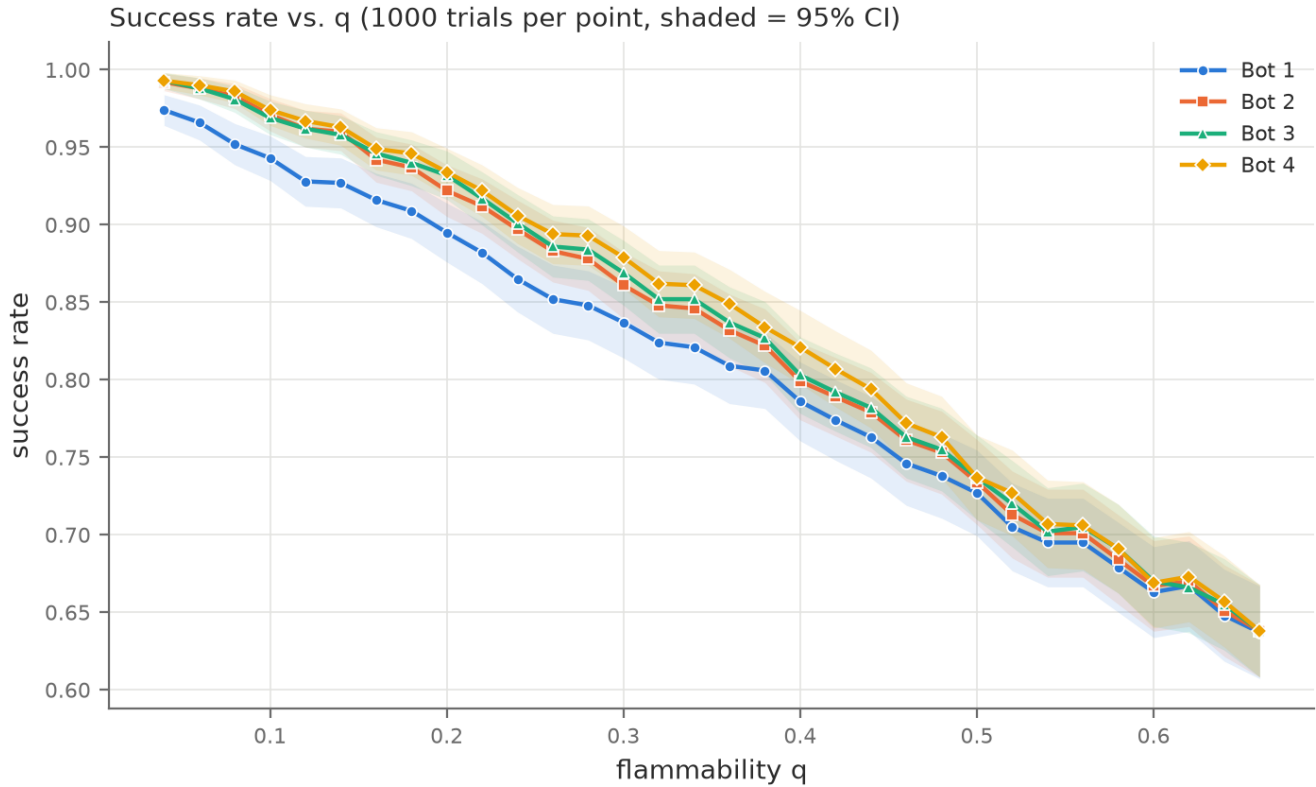


Figure 4. Dense sweep from results/dense.csv. Each point uses 1,000 setups, with the same setup IDs reused across all 32 q values. The curves and bands use the same definitions as Figure 3.

Bot 1 does worst at low and moderate q because its original route can become dangerous as the fire spreads. Bot 2 improves on this by replanning. Bots 3 and 4 gain a bit more by considering danger beyond the cells that are already burning.

At $q = 0.40$, the success rates are 78.6% for Bot 1, 79.9% for Bot 2, 80.3% for Bot 3, and 82.1% for Bot 4. This puts Bot 4 ahead of Bot 3 by 1.8 percentage points. Its clearest gains are around $q = 0.3$ - 0.5 , but it does not win everywhere. At $q = 0.60$, it gets 66.9% compared with Bot 3's 67.0%. At $q = 0.66$, all four get 63.8%. Since some gaps are small, we also compare the bots on the same individual cases.

Why the curves converge at high q

Let d_B be the bot's starting distance to the button and d_F the fire's. We measure both using shortest paths through open cells, ignoring fire. In both sweeps, every failure had d_F at most d_B . None of the bots failed when they started strictly closer to the button than the fire.

We can explain why a strictly closer bot can win by taking a shortest path. To reach the m -th cell on that path, fire needs at least $d_F - (d_B - m)$ moves. When d_F is greater than d_B , this is greater than m . The bot reaches that cell first and stays ahead even if fire spreads as fast as possible.

At $q = 1$, all bots succeed on 97 of the 200 coarse setups, or 48.5%, exactly the cases where the bot started strictly closer. Equal starting distances do not always mean failure: the layout and move-before-fire rule also matter. Being strictly closer is enough to win, but it is not necessary on every ship.

3. Q2: Paired comparisons

Writing responsibility: Yaksh Bhatia

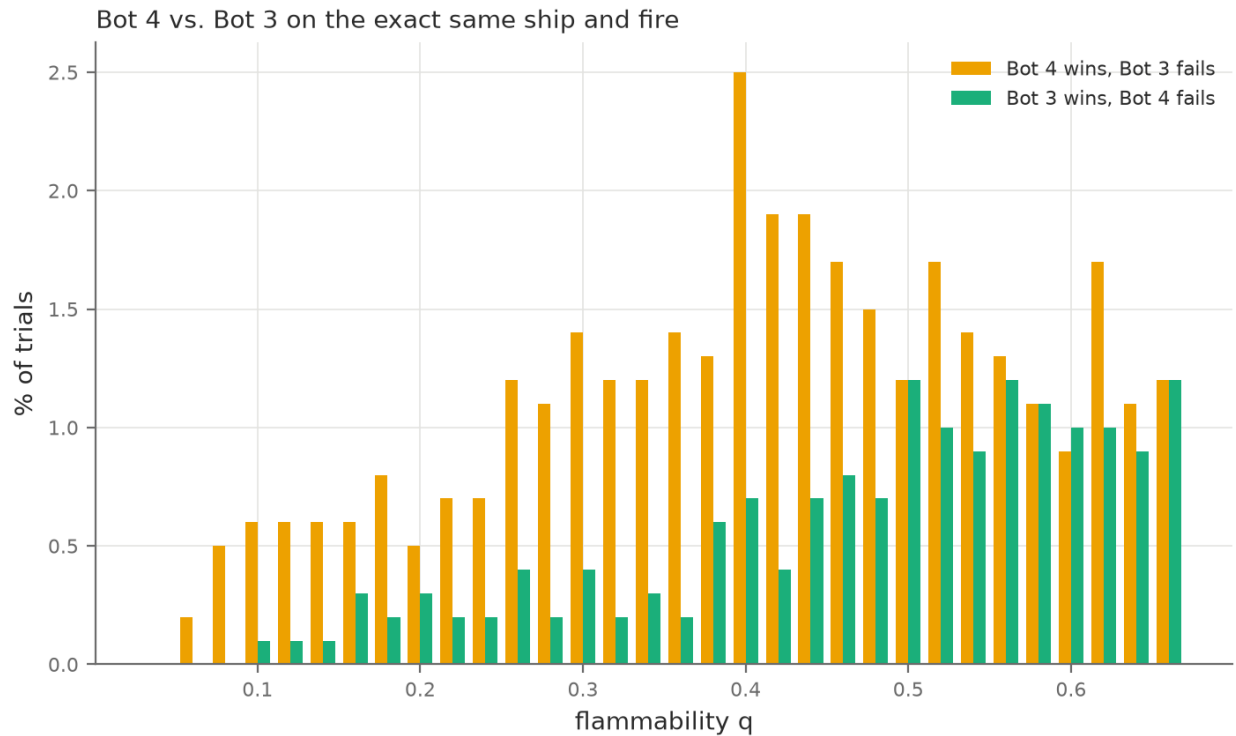


Figure 5. Dense-sweep cases where one bot succeeds and the other fails on the same (q, trial). Each bar gives the percentage of the 1,000 cases at that q. Bot 4 has its biggest net gain over Bot 3 near the middle of the range.

Comparison (X vs. Y)	X only succeeds	Y only succeeds
Bot 4 vs. Bot 3	357	166
Bot 4 vs. Bot 2	427	139
Bot 4 vs. Bot 1	1,005	118
Bot 3 vs. Bot 2	236	139
Bot 3 vs. Bot 1	803	107
Bot 2 vs. Bot 1	599	0

Table 2. Dense sweep, 32,000 matched (q, trial) cases per comparison. Cases where both succeed or both fail are omitted from the two count columns.

These comparisons show when another bot made it through the same fire. Against Bot 3, Bot 4 wins 357 cases and loses 166, giving it 191 more successes, or about 0.60 percentage points over this q grid. Bot 2 wins 599 cases that Bot 1 loses, and we did not see the reverse happen. This describes our test results; it does not prove the same pattern holds for every layout.

The handoff combines the cases in a continuity-corrected McNemar test, giving statistics of about 69, 146, and 25 for Bot 4 vs. Bot 3, Bot 4 vs. Bot 2, and Bot 3 vs. Bot 2. However, the 1,000 trial IDs repeat across q, so all 32,000 pairs cannot be treated as independent. We use the counts to describe the results, but do not rely on the pooled p-values. A stronger overall test would resample whole trial IDs so repeated setups stay together.

4. Q3: What ends a trial?

Writing responsibility: Yaksh Bhatia: Bots 1-2; Vraj Raj: Bots 3-4

Bot	Success	Bot burned	Button burned	Gave up
Bot 1	80.87%	12.21%	6.73%	0.20%
Bot 2	82.74%	2.54%	6.93%	7.80%
Bot 3	83.04%	0.74%	7.58%	8.63%
Bot 4	83.64%	1.53%	7.21%	7.63%

Table 3. Outcome percentages over the dense sweep's 32,000 runs per bot. Percentages use all runs, not just failures, and can sum to slightly above or below 100% because of rounding. Exact counts appear in Appendix A.

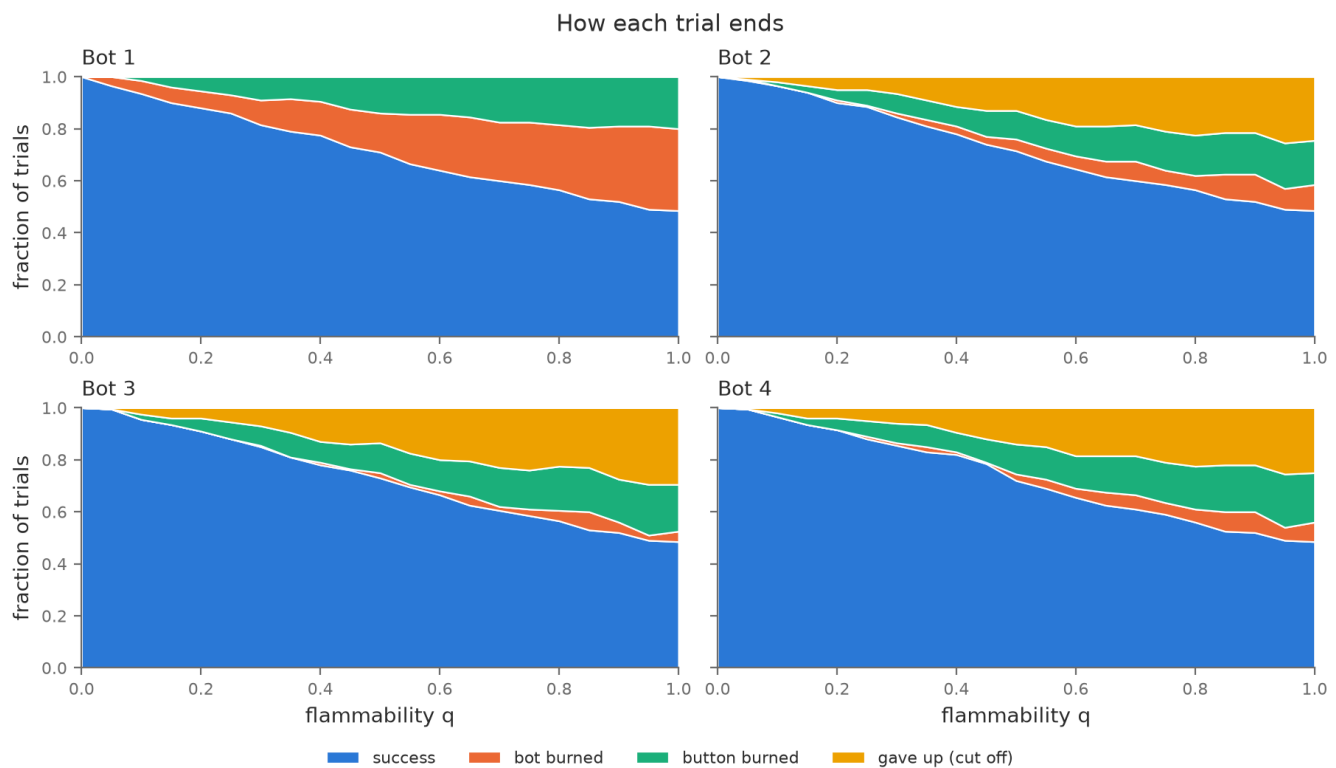


Figure 6. How the coarse-sweep trials end, with 200 trials per q . The colors show outcomes rather than bots. Replanning greatly reduces how often the bot burns, but more failures end with the bot cut off or the button burned.

The outcome tells us how a run ended, but not necessarily which earlier move caused the failure. BURNED can mean the bot walked into fire or was caught by the next spread. BUTTON_BURNED means it lost the target before arriving. GAVE_UP means no non-burning path remained from its position. A bot that gives up might have burned later if it kept moving, so these categories are not separate, independent risks.

We chose the following examples because one bot won and another lost; they are not a random sample of runs. They show that another tested strategy worked on the same fire, but that does not mean it could guarantee success in advance. Each picture shows the fire when that bot's run ended. A blue path can cross orange cells because the bot may have passed through before they burned.

4. Q3: Bots 1 and 2 - reacting to new fire

Writing responsibility: Yaksh Bhatia



Figure 7. Same ship, placement, and fire seed: $q = 0.44$, trial 983. Left: Bot 1 burns at $t = 14$. Right: Bot 2 reaches the button at $t = 45$. Hollow circle = start, blue line = route, filled blue dot = final position, green square = button, orange = fire at that run's stopping time.

Bot 1: the original route can become unsafe

Bot 1 burns in 12.21% of dense-sweep runs, more than any other bot. It keeps following its original path even as the fire gets closer. In Figure 7, that path ends in the growing fire, while Bot 2 moves farther left and reaches the button. Replaying this case shows that Bot 1's final cell catches fire during its last fire update. It did not step into an already burning cell in this example.

Bot 2's success shows that a different strategy could win this case. Across the dense sweep, it wins 599 cases that Bot 1 loses, with no cases going the other way. Still, we cannot say every Bot 1 failure was avoidable. Its 64 GAVE_UP runs (0.20%) have no initial route around the fire cell, so replanning cannot create a legal path.

Bot 2: current safety is not future safety

Bot 2 never chooses a cell that is already burning. Its 2.54% burned rate comes from fire reaching the cell after the bot moves. It can still choose a path right next to fire because BFS does not consider whether that path might ignite on the next update.

Bot 2 more often loses by getting cut off (GAVE_UP, 7.80%) or arriving too late (BUTTON_BURNED, 6.93%). Fire can block the route ahead or reach the button first. Table 2 shows that Bots 3 and 4 win some cases Bot 2 loses, so looking ahead can help. But these totals do not tell us exactly which earlier move would have saved a particular run.

4. Q3: Bot 3 - safety can cost time

Writing responsibility: Vraj Raj



Figure 8. Same setup at $q = 0.40$, trial 358. Left: Bot 3 gives up at $t = 52$. Right: Bot 4 reaches the button at $t = 46$. The button is near the upper-left side of the displayed routes. The pictures show different stopping times, not different initial fires.

Bot 3 has the lowest burned rate, 0.74%. It first looks for a path that avoids current fire and all adjacent cells. If it finds one, its next cell cannot catch fire during the immediate update. If there is no buffered path, it falls back to Bot 2's rule and can still get burned.

Avoiding burns does not give Bot 3 the highest overall success rate. In the dense sweep, it has the most GAVE_UP outcomes (8.63%) and BUTTON_BURNED outcomes (7.58%). The buffer can send it on a longer route or make it change direction while fire keeps spreading. This fits the idea that caution can waste time, but the totals alone do not show how much each detour contributed.

Figure 8 shows a case where another bot could avoid the failure. Bot 3 ends to the right of the fire, cut off from a button that has not burned yet. Bot 4 reaches it sooner on the same setup. Its record shows that FE!N started at $t = 14$. The switch may have helped, but to know how much, we would need to rerun this case with and without that switch.

The trade-off Bot 4 accepts

Compared with Bot 3, Bot 4 lowers cut-offs from 8.63% to 7.63% and button burns from 7.58% to 7.21%. At the same time, it raises bot burns from 0.74% to 1.53%. It accepts more danger to avoid losing time, and that trade-off gives it a small overall gain. Table 2 and Figure 5 show both the wins and losses. The next example shows where the trade-off goes wrong.

4. Q3: Bot 4 - a risk estimate can mislead

Writing responsibility: Vraj Raj



Figure 9. Same setup at $q = 0.48$, trial 392. Left: Bot 4 loses when the button burns at $t = 30$. The burned button is orange with a green outline. Right: Bot 3 reaches the button at $t = 23$. This is a button-deadline failure, not a direct burn of Bot 4.

Bot 4 goes around the lower-right side of the fire, while Bot 3 takes a shorter route closer to it. Bot 4 enters FE!N at $t = 7$, but switching to shortest paths cannot recover the time it already spent or move it back to an earlier position. Bot 3's success shows that a better strategy was available for this particular fire.

We cannot tell from this example which part of Bot 4 caused the bad choice. The independence assumption, limited horizon, search shortcut, and permanent switch could all matter. What we can see is that accounting for the button's deadline did not stop every harmful detour. A useful next step would be to test these parts separately and compare predicted success chances with repeated fire simulations.

Bot 4 loses 166 cases that Bot 3 wins. Some losses can come from taking a risk that turns out badly, while others can come from a poor prediction or an unnecessarily long route. Its higher overall burned rate fits the first explanation. Figure 9 shows the second kind of problem: the button burns while Bot 4 is still trying to reach it, rather than the bot itself burning.

What can we say about unavoidable losses?

In both sweeps, every failure starts with fire at least as close to the button as the bot. That does not mean all of those failures were unavoidable. At moderate q , fire can spread slowly enough for a good route to work, and the paired examples show that the choice of strategy still matters. We did not calculate an ideal strategy for each case or count how many losses no bot could avoid. Some were likely unavoidable, but we do not know what fraction.

5. Q4: Toward an ideal bot

Writing responsibility: Yaksh Bhatia: information and planning; Vraj Raj: computation and rushing

Using the available information

An ideal bot would use the full layout, current fire, its own position, the button position, and q . It would also account for how much time it has to choose a move. The main question is whether it can still reach each route cell and the button in time. It should predict possible future fires without looking at the actual fire's random stream.

One improvement would be to search using both position and arrival time. That would let us keep an earlier arrival even if it has a higher cost so far. A fully optimal strategy would also respond to the fire states it observes later. This makes the problem much bigger than finding one shortest path on a fixed map.

We could use Monte Carlo look-ahead to simulate possible fires instead of assuming neighboring fire events are independent. The `fork_fire` function copies the current fire state and takes a separate random generator. For each promising move, we could try several possible futures and compare how often the bot reaches the button. Those simulations need their own random seeds so they do not reveal the actual future fire.

Deciding how much time to spend planning

Checking every possible future fire would take too long. A more practical bot could start with a BFS route and compare a few alternative moves. It could run more simulations when the choices look close, reuse calculations from the previous step, and focus on narrow passages where a wrong move could trap it. It would also need to stop planning in time to make its move.

Our timing results show the cost of extra planning. Bot 4 takes about 9 times as long per step as Bot 3, but its average gain over the dense q grid is about 0.60 percentage points. The gain is larger at some middle q values. Whether this is worth it depends on how quickly the fire spreads compared with the time spent deciding, which our simulator does not model.

When making a break for it is sensible

If the bot is strictly closer to the button than the current fire is, measured through open cells, it can take a shortest path and stay ahead under the one-cell-per-step rule. It does not need extra predictions in that case. Rushing can also make sense when a detour would leave the button exposed for too long, or when predictions far ahead are too uncertain to justify the delay.

FE!N Mode is our attempt at this balance. It compares the careful route with the shortest current route and decides whether the detour is worth it. An improved version could reconsider that choice after seeing new fire and skip forecasts that no longer affect the move. To test this fairly in real time, we would need fire to keep spreading while the bots think and give each bot the same computation budget.

Based on our results, the ideal bot should be careful when there is time to go around the fire, but recognize when getting to the button quickly matters more. Bot 4 moves in that direction, although its extra planning still makes the wrong choice in some cases.

6. Contributions and reproducibility

Writing responsibility: Yaksh Bhatia and Vraj Raj

We split both the coding and writing as shown below. This follows our project handoff and TASKS.md.

Area	Yaksh Bhatia	Vraj Raj
Infrastructure	ship.py, render.py, demo.py	fire.py, sim.py
Bots 1-3	BFS helper, Bot 1, Bot 2	Bot 3 and bot tests
Bot 4	Fire-risk model, arrival danger, path survival, risk-weighted search	FE!N trigger, mode switching, configuration, logging, trigger bug fix
Figures	Success curves and risk map	Outcome breakdown, paired chart, failure examples
Writeup	Ship setup; Q1 risk model/search; Q2; Q3 Bots 1-2; Q4 information	Fire/simulation setup; Q1 FE!N; Q3 Bots 3-4; Q4 computation
Tuning	risk_weight and horizon sweeps, charts, and discussion	fein_margin and max_detour_ratio sweeps, charts, and discussion
Final pass	PDF assembly and review of own sections	Proofreading, data checks, and review of own sections

Where the results came from

We used the assignment PDF for the rules and analysis questions. WRITEUP_HANDOFF.md records the division of work, bug fix, and benchmarks, and the Python files show how we implemented the bots. We checked the success tables and paired counts against results/coarse.csv and results/dense.csv, and replayed the six example runs from their saved seeds. Figures 1-9 show the main and tuning results. The four tuning CSVs add 48,000 run records. This report uses github.com/Ybhatia18/IntroAIproj1 at revision c9bda7d.

How to rerun the experiments

To rerun the experiments, use Python with NumPy and Matplotlib and run the commands below from the project folder. They include the dimension, seed, and worker settings from the handoff. TASKS.md has the plot and tuning commands, and tuning/ contains the completed tuning data and graphs.

```
python experiments.py sweep --D 40 --seed 440 --workers 8 --trials 200
--out results/coarse.csv
```

```
python experiments.py sweep --D 40 --seed 440 --workers 8 --q-start 0.04
--q-stop 0.66 --q-step 0.02 --trials 1000 --out results/dense.csv
```

Enter each two-line command above as one command. The first uses the default q range of 0 to 1 in steps of 0.05. The package contains the code, tests, graphs, raw data, and this report as a PDF and editable Markdown. All six CSVs are also attached inside the PDF. If your reader does not show them, use the package's results/ and tuning/results/ folders.

Appendix A. Full-range data

Each entry is the fraction of successful runs out of 200 for that bot and q. Source: results/coarse.csv. Multiply by 200 to get the number of successes.

q	Bot 1	Bot 2	Bot 3	Bot 4
0.00	1.000	1.000	1.000	1.000
0.05	0.965	0.985	0.995	0.995
0.10	0.935	0.965	0.955	0.965
0.15	0.900	0.940	0.935	0.935
0.20	0.880	0.900	0.910	0.915
0.25	0.860	0.885	0.880	0.880
0.30	0.815	0.845	0.850	0.855
0.35	0.790	0.810	0.810	0.830
0.40	0.775	0.780	0.780	0.820
0.45	0.730	0.740	0.760	0.785
0.50	0.710	0.715	0.730	0.720
0.55	0.665	0.675	0.695	0.690
0.60	0.640	0.645	0.665	0.655
0.65	0.615	0.615	0.625	0.625
0.70	0.600	0.600	0.605	0.610
0.75	0.585	0.585	0.585	0.590
0.80	0.565	0.565	0.565	0.560
0.85	0.530	0.530	0.530	0.525
0.90	0.520	0.520	0.520	0.520
0.95	0.490	0.490	0.490	0.490
1.00	0.485	0.485	0.485	0.485

Exact outcome counts for both main sweeps

Sweep	Bot	Success	Burned	Button burned	Gave up
Coarse	Bot 1	3,011	695	494	0
Coarse	Bot 2	3,055	178	409	558
Coarse	Bot 3	3,074	70	435	621
Coarse	Bot 4	3,090	126	440	544
Dense	Bot 1	25,877	3,907	2,152	64
Dense	Bot 2	26,476	812	2,217	2,495
Dense	Bot 3	26,573	238	2,427	2,762
Dense	Bot 4	26,764	489	2,306	2,441

Each bot has 4,200 coarse and 32,000 dense runs. Fire begins at least as close to the button in 2,163 coarse and 16,224 dense cases; these include repeated setup IDs across q. No bot fails in the strictly-closer cases. Neither sweep has a TIMEOUT. Bot 4 activates FE!N in 888 coarse and 4,242 dense runs.

Appendix B. Dense-range data

Each entry is the fraction of successful runs out of 1,000 for that bot and q. Source: results/dense.csv. Multiply by 1,000 to get the number of successes.

q	Bot 1	Bot 2	Bot 3	Bot 4
0.04	0.974	0.992	0.993	0.993
0.06	0.966	0.988	0.988	0.990
0.08	0.952	0.983	0.981	0.986
0.10	0.943	0.971	0.969	0.974
0.12	0.928	0.962	0.962	0.967
0.14	0.927	0.960	0.958	0.963
0.16	0.916	0.942	0.946	0.949
0.18	0.909	0.937	0.940	0.946
0.20	0.895	0.922	0.932	0.934
0.22	0.882	0.912	0.917	0.922
0.24	0.865	0.897	0.901	0.906
0.26	0.852	0.883	0.886	0.894
0.28	0.848	0.878	0.884	0.893
0.30	0.837	0.861	0.869	0.879
0.32	0.824	0.848	0.852	0.862
0.34	0.821	0.846	0.852	0.861
0.36	0.809	0.832	0.837	0.849
0.38	0.806	0.822	0.827	0.834
0.40	0.786	0.799	0.803	0.821
0.42	0.774	0.789	0.792	0.807
0.44	0.763	0.779	0.782	0.794
0.46	0.746	0.761	0.763	0.772
0.48	0.738	0.753	0.755	0.763
0.50	0.727	0.734	0.737	0.737
0.52	0.705	0.713	0.720	0.727
0.54	0.695	0.701	0.702	0.707
0.56	0.695	0.701	0.705	0.706
0.58	0.679	0.684	0.691	0.691
0.60	0.663	0.667	0.670	0.669
0.62	0.667	0.670	0.666	0.673
0.64	0.648	0.651	0.655	0.657
0.66	0.637	0.638	0.638	0.638

The overall rates give each tested q equal weight. The coarse sweep uses a subset of the dense setup IDs, and tuning reuses the first 500 dense IDs, so these are not independent test sets. All six unchanged CSVs are attached to the PDF and included in the package. They record q, trial, bot, tuning parameter/value, outcome, steps, starting distances to the button, and FEIN activation time.